

Lekce 6: Instalace a konfigurace Cypressu

1. Osnova obsahu

A. Úvod do instalace Cypressu

- **Proč je instalace důležitá:**
 - Než začnete psát Cypress testy, musíte mít správně nastavené prostředí, abyste zajistili kompatibilitu a hladký průběh testů.
- **Předpoklady pro instalaci:**
 - Ve vašem systému musí být nainstalován Node.js.
 - Základní znalost správců balíčků jako `npm` nebo `yarn`.

1. Node.js nainstalován:

- Cypress vyžaduje Node.js pro správu své instalace a závislostí.
- Nainstalujte Node.js z oficiálních stránek: <https://nodejs.org/>.
- Ověřte instalaci pomocí:

```
node -v  
npm -v
```

2. Správce balíčků:

- Node.js obsahuje `npm` (Node Package Manager) ve výchozím nastavení. Můžete však využít i `yarn`, `pnpm` nebo jiné.

3. Inicializovaný JavaScript projekt:

- Váš projekt by měl mít soubor `package.json`, který vytvoříte pomocí `npm init` nebo `yarn init`. Tento soubor eviduje závislosti jako Cypress a další metadata projektu.

4. Základní orientace v příkazové řádce:

- Pro instalaci a spuštění Cypressu budete muset spouštět příkazy v terminálu.

Co je npm a jak funguje?

Co je npm?

- `npm` (Node Package Manager) je nástroj dodávaný s Node.js, sloužící k správě JavaScriptových balíčků a závislostí.
- Pomáhá vývojářům:
 - Instalovat knihovny a nástroje.
 - Sdílet vlastní balíčky s komunitou.

- Spravovat závislosti specifické pro projekt nebo globální závislosti.

Jak npm funguje:

1. Instalace balíčků:

- Instalace balíčku lokálně (pro konkrétní projekt):

```
npm install package-name
```

- Instalace balíčku globálně (dostupný ve všech projektech):

```
npm install -g package-name
```

2. Správa závislostí:

- Instalované balíčky jsou uvedeny v souboru `package.json` v sekcích `dependencies` nebo `devDependencies`.

3. Automatizace skriptů:

- `npm` může spouštět vlastní skripty definované v `package.json`:

```
{
  "scripts": {
    "start": "node app.js",
    "test": "cypress open"
  }
}
```

- Spuštění skriptu:

```
npm run test
```

Rozdíl mezi npm, yarn, bum a pnpm

Vlastnost	npm	yarn	bun	pnpm
Rychlost	Průměrná	Rychlejší než npm	Nejrychlejší	Rychlejší než npm/yarn
Využití disku	Vyšší (duplicitní balíčky)	Průměrné	Efektivní	Nízké (používá hardlinky pro eliminaci duplicit)

Vlastnost	npm	yarn	bun	pnpm
Strom závislostí	Plochý strom	Plochý strom	Plochý strom	Hierarchický strom (lepší izolace)
Offline režim	Omezený	Podporováno	Podporováno	Podporováno
Adopce	Široce používán	Oblíbený ve větších projektech	Nový (nastupující)	Stoupající popularita v enterprise
Nejlepší využití	Obecné	Velké nebo výkonnostní projekty	Rychlé vývojové workflow	Projekty vyžadující efektivní úložiště

B. Instalace krok za krokem

1. Inicializujte projekt:

- Vytvořte novou složku projektu.
- Spusťte `npm init` nebo `yarn init` pro vytvoření souboru `package.json`.

2. Instalujte Cypress:

- Použijte příkaz `npm install cypress --save-dev` nebo `yarn add cypress --dev`.

3. Ověřte instalaci:

- Spusťte `npx cypress open` nebo `yarn cypress open` pro otevření Cypress Test Runneru.

Co je npm init ?

Účel:

`npm init` inicializuje nový Node.js projekt vytvořením souboru `package.json`, který slouží jako konfigurační soubor pro správu závislostí a skriptů projektu.

Jak to funguje:

1. Spusťte `npm init` ve složce projektu:

```
npm init
```

2. Odpovězte na výzvy (název projektu, verze, popis atd.), nebo použijte parametr `-y` pro výchozí hodnoty:

```
npm init -y
```

3. Vznikne soubor `package.json`, který obsahuje například tato data:

```
{
  "name": "my-project",
  "version": "1.0.0",
  "scripts": {},
  "dependencies": {},
  "devDependencies": {}
}
```

C. Přehled struktury složek Cypressu

- **Výchozí adresáře:**
 - `cypress/fixtures` : Uchovává JSON soubory pro testovací data.
 - `cypress/e2e` : Sem pište testovací soubory.
 - `cypress/plugins` : Rozšiřitelnost Cypressu.
 - `cypress/support` : Ukládání znovupoužitelných utilit a globální konfigurace.
- **Význam jednotlivých adresářů:**
 - **fixtures** : Centralizace mock dat pro testy.
 - **integration** : Organizace testovacích případů dle funkcionalit.
 - **support** : Přizpůsobení příkazů a konfigurace globálního chování.

Význam složek Cypressu

1. fixtures :

- Uchovávejte mock data ve formátu JSON.
- Používá se pro simulaci odpovědí API nebo zadání vstupních dat pro testy.
- Příklad: `cypress/fixtures/users.json`

```
[
  { "id": 1, "name": "Alice", "role": "admin" },
  { "id": 2, "name": "Bob", "role": "user" }
]
```

- Přístup v testech:

```
cy.fixture('users').then((users) => {
  cy.log(users[0].name); // Vypíše "Alice"
});
```

2. e2e (nebo integration ve starších verzích):

- Uchovávejte skutečné testovací soubory, obvykle rozdělené dle funkcionalit.
- Příklad:

```
cypress/integration/login.spec.js
cypress/integration/dashboard.spec.js
```

3. plugins :

- Rozšiřte funkčnost Cypressu, například úpravou chování prohlížeče nebo konfigurací pluginů.
- Příklad:

```
module.exports = (on, config) => {
  // Úprava konfigurace nebo nastavení event listenerů
};
```

4. support :

- Obsahuje znovupoužitelný kód jako vlastní příkazy nebo globální konfigurace.
- Příklad: Přidání vlastního přihlašovacího příkazu v `cypress/support/commands.js` :

```
Cypress.Commands.add('login', (username, password) => {
  cy.get('[data-testid="username"]').type(username);
  cy.get('[data-testid="password"]').type(password);
  cy.get('[data-testid="login-button"]').click();
});
```

D. Konfigurační soubory

- **Přehled souboru `cypress.config.js` :**

- Nastavení základní URL, velikosti viewportu, timeoutů a další.
- Příklad:

```
const { defineConfig } = require('cypress');

module.exports = defineConfig({
  e2e: {
    baseUrl: 'http://localhost:3000',
    viewportWidth: 1280,
    viewportHeight: 720,
    defaultCommandTimeout: 8000,
    supportFile: 'cypress/support/index.js',
  },
});
```

Co je `baseUrl` v kontextu Cypressu?

Definice:

`baseUrl` je konfigurační vlastnost v `cypress.config.js`, která nastavuje výchozí kořenovou URL pro testovanou aplikaci. Umožňuje zjednodušit testovací kód, protože není potřeba opakovaně uvádět celou URL v příkazech `cy.visit()`.

Příklad:

- V `cypress.config.js`:

```
module.exports = {  
  e2e: {  
    baseUrl: 'http://localhost:3000',  
  },  
};
```

- Použití v testech:

```
cy.visit('/login'); // Rozhodí se na 'http://localhost:3000/login'
```

Výhody:

- Snižuje redundanci v testovacích souborech.
- Usnadňuje změnu root URL (například při přesunu z `localhost` na staging server).
- **Nastavení environmentálních proměnných:**
 - Použijte `env` v konfiguračním souboru pro citlivá data nebo opakovaně využívané hodnoty.
 - Příklad:

```
module.exports = defineConfig({  
  env: {  
    apiUrl: 'https://api.example.com',  
  },  
});
```

- Přístup v testech pomocí `Cypress.env('apiUrl')`.

Vlastnost `env` v `cypress.config.js`

Definice:

Vlastnost `env` v `cypress.config.js` ukládá proměnné specifické pro prostředí, jako jsou API

endpointy, údaje pro přihlášení nebo různé příznaky. Tyto hodnoty lze v testech získat pomocí `Cypress.env()` .

Příklad:

- V `cypress.config.js` :

```
module.exports = {
  e2e: {
    env: {
      apiUrl: 'https://api.example.com',
      userRole: 'admin',
    },
  },
};
```

- Použití v testech:

```
describe('API Tests', () => {
  it('should fetch data from the API', () => {
    const apiUrl = Cypress.env('apiUrl');
    cy.request(`${apiUrl}/users`).then((response) => {
      expect(response.status).to.eq(200);
    });
  });
});
```

Výhody:

- Centralizace hodnot specifických pro prostředí, což zjednodušuje jejich aktualizaci.
- Udržení citlivých údajů (například API klíčů) mimo testovací kód.

E. Spouštění Cypress testů

- **Open režim:**
 - Použijte `npx cypress open` pro spuštění Test Runner UI.
- **Headless režim:**
 - Použijte `npx cypress run` pro provedení testů v terminálu.
 - Vhodné pro CI/CD pipelines.
- **Příklad příkazu:**

```
npx cypress run --spec cypress/integration/login.spec.js
```

npx je nástroj dodávaný s **npm** (od verze 5.2.0) a slouží ke spouštění Node.js balíčků přímo z příkazové řádky bez nutnosti jejich globální instalace. Umožňuje spustit binárky balíčků, které jsou buď nainstalované lokálně v projektu, nebo je stáhne přímo z npm registru.

Proč používat **npx** ?

1. Dočasné spuštění:

- S **npx** můžete spustit balíček dočasně bez jeho trvalé instalace do systému.
- Například:

```
npx create-react-app my-app
```

Tento příkaz spustí balíček `create-react-app` bez nutnosti globální instalace.

2. Vyhněte se globálním instalacím:

- Místo instalace balíčku globálně (`npm install -g some-package`) získá a spustí **npx** balíček napřímo.
- Tím udržíte svůj globální systém čistý a vyvarujete se konfliktům verzí.

3. Spouštění lokálně nainstalovaných balíčků:

- Pokud je balíček nainstalovaný lokálně v projektu (`node_modules`), **npx** jej automaticky spustí bez nutnosti psát celou cestu.
- Příklad:

```
npx cypress open
```

Tento příkaz spustí Cypress z lokální složky `node_modules` , pokud je instalován.

4. Spuštění konkrétních verzí:

- Můžete uvést konkrétní verzi balíčku:

```
npx some-package@1.2.3
```

5. Scaffolding a generátory:

- **npx** se hojně využívá pro generování nových projektů nebo konfiguračních souborů bez nutnosti globální instalace nástroje.
- Příklad:

```
npx eslint --init
```

Jak **npx** funguje?

- Když spustíte příkaz pomocí **npx** :
 - i. Zkontroluje, zda balíček existuje lokálně v `node_modules` .

- ii. Pokud není nalezen, dočasně stáhne balíček a spustí jej.
- iii. Po spuštění balíček (pokud byl stažen) odstraní.

Kdy použít npx vs npm ?

Vlastnost	npm	npx
Instalace	Instaluje balíček lokálně nebo globálně.	Spustí balíček bez instalace.
Použití	Je třeba balíček nejdříve nainstalovat.	Nevyžaduje instalaci; provede přímo.
Globální balíčky	Často se používá ke správě globálních nástrojů.	Nahrazuje nutnost globální instalace.
Jednorázové použití	Není vhodné bez instalace.	Ideální pro jednorázové spuštění.

Příklady použití npx

1. Spuštění Cypressu bez globální instalace:

```
npx cypress open
```

2. Vytvoření nové React aplikace:

```
npx create-react-app my-app
```

3. Lintování souborů bez globální instalace ESLintu:

```
npx eslint myfile.js
```

4. Spuštění konkrétní verze balíčku:

```
npx some-package@2.0.0
```

5. Dočasné spuštění pro testování:

- Spusťte `npx cowsay` pro zábavu:

```
npx cowsay "Hello, World!"
```

`npx` je univerzální nástroj, který zjednodušuje spouštění balíčků a pomáhá udržet systém čistý bez zbytečných globálních instalací, což ocení zejména vývojáři často pracující s JavaScriptovými nástroji a frameworky.

F. Osvědčené postupy při nastavování

- **Mějte Cypress aktuální:**
 - Pravidelně aktualizujte Cypress kvůli novým funkcím a opravám chyb.
- **Používejte verzovací systém pro konfigurační soubory:**
 - Sledujte změny v `cypress.config.js` a dalších souborech nastavení.
- **Definujte základní URL:**
 - Nastavte základní URL v konfiguračním souboru pro zjednodušení příkazů jako `cy.visit()`.

2. Praktické aktivity

A. Cvičení na instalaci Cypressu

- **Cíl:**
 - Provést studenty instalací Cypressu v jejich prostředí.
- **Kroky:**
 - i. Nainstalujte Node.js, pokud již není nainstalován.
 - ii. Inicializujte nový projekt pomocí `npm init`.
 - iii. Nainstalujte Cypress příkazem `npm install cypress --save-dev`.
 - iv. Spusťte `npx cypress open` a prozkoumejte výchozí strukturu složek.
- **Výsledek:**
 - Studenti by měli úspěšně otevřít Cypress Test Runner.

B. Cvičení na nastavení konfiguračního souboru

- **Cíl:**
 - Upravte soubor `cypress.config.js` pro přidání základní URL a nastavení viewportu.
- **Kroky:**
 - i. Otevřete soubor `cypress.config.js`.
 - ii. Přidejte následující:

```
module.exports = {
  e2e: {
    baseUrl: 'http://localhost:3000',
    viewportWidth: 1366,
    viewportHeight: 768,
  },
};
```

iii. Uložte soubor a znovu spusťte `npx cypress open`.

- **Výsledek:**

- Studenti by měli být schopni navštívit základní URL příkazem `cy.visit('/')` ve svých testech.

3. Možné otázky studentů

1. **Co se stane, když nemám nainstalovaný Node.js?**

- **Odpověď:** Cypress vyžaduje instalaci Node.js, protože jej spravuje npm (Node Package Manager). Studenti si musí stáhnout a nainstalovat Node.js před pokračováním.

2. **Musím instalovat Cypress globálně?**

- **Odpověď:** Ne, Cypress se obvykle instaluje lokálně v rámci projektu pro lepší správu verzí a reprodukovatelnost.

3. **Jaký je rozdíl mezi open režimem a headless režimem Cypressu?**

- **Odpověď:** Open režim spustí interaktivní Cypress Test Runner, kde studenti vidí testy vizuálně. Headless režim běží testy v terminálu bez UI, což je vhodné pro automatizační pipeline.

4. **Proč bych měl používat základní URL v konfiguraci?**

- **Odpověď:** Nastavení základní URL zjednodušuje příkazy v testech. Místo `cy.visit('http://localhost:3000/login')` stačí napsat `cy.visit('/login')`.

5. **Mohu používat Cypress v existujících projektech?**

- **Odpověď:** Ano, Cypress lze přidat do libovolného JavaScriptového projektu instalací jako vývojovou závislost.

4. Doplnkové materiály

A. Oficiální dokumentace:

- [Průvodce instalací Cypressu](#)
- [Konfigurace Cypressu](#)

B. Tutoriály a články:

- [Struktura složek Cypressu a nastavení](#)
- [Začátečnický průvodce instalací Cypressu](#)

C. Video:

- **Traversy Media:** Cypress Crash Course
- **The Net Ninja:** Úvod do testování pomocí Cypressu

D. Tréninkové platformy:

- Nastavte Cypress v jednoduché aplikaci TodoMVC nebo demo stránce jako <http://todomvc.com> .

5. Doporučený rozvrh lekce pro 3 hodiny

1. hodina: Úvod a instalace (60 minut)

- **Instalace Cypressu (30 minut):**
 - Postupné vedení studentů instalací.
 - Ověření nastavení pomocí `npm cypress open` .
- **Přehled struktury složek (20 minut):**
 - Vysvětlení účelu jednotlivých složek.
 - Diskuze, kam psát testy a ukládat mock data.
- **Dotazy a odpovědi (10 minut):**
 - Odpovědi na dotazy k instalaci.

2. hodina: Konfigurace Cypressu (60 minut)

- **Vytváření a úprava `cypress.config.js` (30 minut):**
 - Nastavení základní URL, velikosti viewportu a custom environment proměnných.
 - Diskuze o významu konfiguračních souborů.
- **Praktická aktivita (20 minut):**

- Úprava konfiguračního souboru a spuštění Test Runneru pro ověření změn.
- **Dotazy a odpovědi (10 minut):**
 - Odpovědi na dotazy ohledně konfigurace.

3. hodina: Spouštění Cypress testů (60 minut)

- **Open a headless režim (20 minut):**
 - Ukázka obou režimů a vysvětlení jejich využití.
- **Prozkoumání ukázkových testů (30 minut):**
 - Spuštění výchozích ukázkových testů v Cypressu a vysvětlení vestavěných funkcí.
 - Diskuze o struktuře vlastních testů ve složce `integration`.
- **Shrnutí a dotazy (10 minut):**
 - Opakování kroků instalace a konfigurace.
 - Doplnění odpovědí na závěrečné dotazy a příprava na další lekci.

6. Další doporučení

Interaktivní ukázky:

- Ukažte studentům řešení běžných instalačních potíží (například oprávnění, chybějící Node.js).
- Živě editujte `cypress.config.js` a ukažte okamžité změny.

Podporujte aktivní účast:

- Nechte studenty instalovat Cypress samostatně a pomáhejte si navzájem s nastavením.
- Povzbudte experimentování s nastavením jako jsou `timeouts` a velikosti `viewportu`.

Poskytujte jasné pokyny:

- Krok za krokem pro instalaci i konfiguraci.
- Tipy pro organizaci složky `integration` pro škálovatelné testovací případy.

Důraz na praktičnost:

- Používejte srozumitelné příklady, například nastavení základní URL pro ukázkovou aplikaci.
- Vysvětlíte, jak správné nastavení ovlivňuje udržitelnost a škálovatelnost automatizovaných testovacích projektů.